

Jazz Standards Repertoire Analytics

Documento de visión técnica — qué construimos, con qué stack y por qué

Mathias Berg · Stuttgart · Junio 2026 · Estado: Semana 1 (scaffold) · Repo: github.com/mberger/jazz-standards-analytics

01 Resumen ejecutivo

Proyecto **end-to-end de analytics engineering** que ingesta, modela y visualiza datos sobre el **canon de los jazz standards**: qué temas forman el repertorio, quién los grabó y cómo evolucionó a lo largo de las décadas. Es la pieza central de tu portfolio porque demuestra el **modern data stack completo** —no solo análisis o dashboards— y conecta tus dos disciplinas diarias: los datos y la guitarra de jazz.

La diferencia con un proyecto de Data Analyst tradicional: acá no recibís una tabla limpia para analizar. **Construís toda la cadena**: extraés los datos crudos, los almacenás, los transformás con código testeado y documentado, y recién entonces los visualizás. Ese recorrido es exactamente lo que distingue al *analytics engineer*.

02 Objetivo

- **Demostrar el stack de punta a punta** — ingesta → almacenamiento → transformación → visualización— como evidencia concreta del salto de Data Analyst a Analytics Engineer.
- **Responder preguntas reales** sobre el repertorio (cuáles son los standards más grabados, cómo evoluciona la huella de un compositor, qué artistas comparten repertorio).
- **Construirlo en público**: commits frecuentes, README vivo y un write-up en el blog, para que un reclutador vea el proceso, no solo el resultado.

03 Alcance

El alcance está deliberadamente acotado para llegar a un resultado funcional rápido. El principio rector está en el log de decisiones:

Principio de alcance

“Un MVP que funciona de punta a punta vale más que un dataset más grande sin marts.” Primero el pipeline completo con ~100 standards; la expansión viene después.

Dentro del MVP	Fuera del MVP (más adelante)
~100 standards top de JazzStandards.com	Expandir a ~500 standards una vez que todo funcione
Metadata de grabaciones desde Spotify	Audio features avanzados (tempo, tonalidad, energía)
Pipeline completo hasta un dashboard publicado	Análisis de redes de músicos / sidemen

04 Fuentes de datos

Fuente	Cómo se accede	Qué aporta
JazzStandards.com	Scraping (requests + beautifulsoup4)	El canon: título, compositor, año, show o película original.
Spotify Web API	Cliente spotipy (OAuth, credenciales en .env)	Grabaciones por standard: artistas, álbumes y fechas.

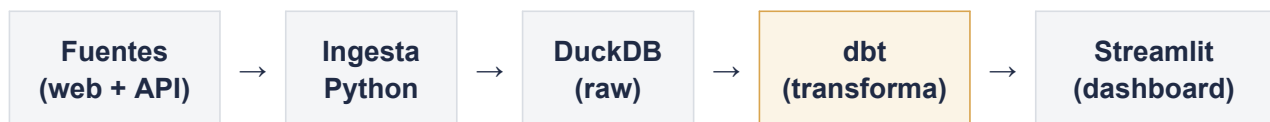
Las credenciales de Spotify se obtienen en el dashboard de developers y nunca se commitean: viven en **.env** (ignorado por git), con **.env.example** como plantilla.

05 Stack tecnológico

Capa	Herramienta	Por qué
Ingesta	Python (requests, beautifulsoup4, spotipy)	Lenguaje estándar para extraer datos de webs y APIs.
Almacenamiento	DuckDB	Base analítica en un solo archivo, sin servidor ni infraestructura.
Transformación	dbt-duckdb	Modela los datos con SQL versionado, testeado y documentado.
Visualización	Streamlit (+ plotly, pandas)	Dashboard interactivo en Python puro, lee directo de los marts.

06 Arquitectura del pipeline

Los datos fluyen en una sola dirección, de crudo a listo para consumir:



- **Raw schema:** los datos entran a DuckDB tal cual llegan, sin limpiar. Si algo se rompe, siempre podés reconstruir desde el crudo sin volver a scrapear.
- **dbt** hace el trabajo pesado de transformación en tres capas (siguiente sección). Su salida son los **marts**: las tablas finales y limpias.
- **Streamlit** lee solo de los marts; nunca toca los datos crudos. Separación clara de responsabilidades.

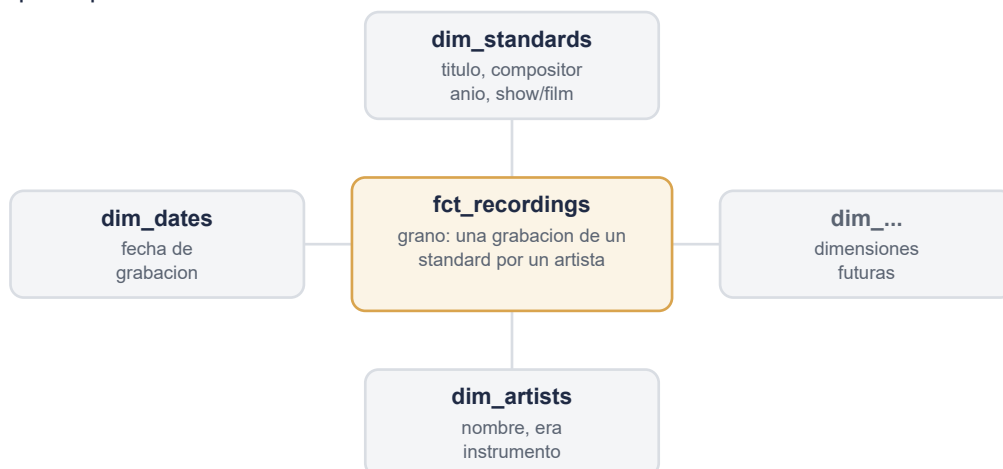
07 Las tres capas de dbt

dbt organiza la transformación en capas, cada una sobre la anterior. Es el estándar de la industria porque hace el SQL legible, testeable y reutilizable —en vez de una consulta gigante imposible de mantener.

Capa	Qué hace
staging	Una vista por cada fuente. Limpia, castea tipos y renombra columnas. Espejo ordenado del crudo.
intermediate	Une fuentes y deduplica versiones de grabaciones (remasters, compilaciones).
marts	El modelo final dimensional (star schema) que consume el dashboard.

08 Modelo de datos: el star schema

El mart se modela como un **esquema en estrella**: una tabla de **hechos** en el centro (cada fila es un evento medible) rodeada de **dimensiones** (el contexto: quién, qué, cuándo). Es el patrón clásico de modelado analítico y el que esperan ver los reclutadores de AE.



- **fct_recordings** (hechos) — grano: una grabación de un standard por un artista. El grano es la decisión de diseño más importante: define qué representa exactamente cada fila.
- **dim_standards / dim_artists / dim_dates** — el contexto descriptivo que rodea a los hechos.
- Todos los modelos llevan **tests** (not_null, unique, relaciones entre tablas) y documentación a nivel columna, renderizada con **dbt docs**.

09 Desafíos principales

Vincular standards con grabaciones (entity resolution)

El reto central de calidad de datos: unir un standard con sus grabaciones en Spotify cuando los nombres son ambiguos, hay covers y versiones en vivo. Una mala vinculación contamina todo el análisis.

Deduplicación de versiones

Remasters, compilaciones y reediciones son la MISMA grabación. La regla para colapsarlas vive en la capa intermediate y se documenta caso por caso.

Límites y autenticación de la Spotify API

Hay rate limits y un flujo OAuth que manejar; la ingesta debe ser resiliente y reanudable.

Scraping frágil

Si JazzStandards.com cambia su HTML, el scraper se rompe. Por eso el crudo se guarda intacto en DuckDB.

Definir bien el grano

Si “una grabación” no está definida con precisión, el fact table queda inconsistente y las métricas mienten.

10 Roadmap

Paso	Entregable	Estado
01 · Design	Alcance, arquitectura y modelo dimensional	☐ hecho 2026-06-10
02 · Ingestion	Scraping + Spotify API → DuckDB raw	pendiente

03 · Staging	Capa staging dbt, sources, tests básicos	pendiente
04 · Marts	Modelos intermedios + star schema + tests de relación	pendiente
05 · Docs & metrics	dbt docs, lineage, métricas (más grabados, longevidad)	pendiente
06 · Dashboard	App Streamlit en Streamlit Community Cloud	pendiente
07 · Write-up	Screenshots, pulido y post en el blog	pendiente

11 Preguntas que responde el proyecto

- ¿Cuáles son los standards más grabados, y coincide ese “canon” con lo que realmente se llama en las jam sessions?
- ¿Cómo evoluciona la huella de un compositor a lo largo de las décadas (Ellington vs. Porter vs. Monk)?
- ¿Qué artistas tienen el mayor solapamiento de repertorio, y quién grabó los temas más raros?

12 Glosario de analytics engineering

Términos que aparecen en el proyecto y que marcan el vocabulario del rol:

Término	Qué significa
Modern data stack	Conjunto de herramientas modulares (ingesta, warehouse, dbt, BI) que reemplazó al ETL monolítico.
ELT	Extract-Load-Transform: cargás el crudo primero y transformás dentro de la base. Lo opuesto al viejo ETL.
dbt	Herramienta que convierte SELECTs de SQL en un pipeline de modelos versionado, testeado y documentado.
Star schema	Modelo con una tabla de hechos central y dimensiones alrededor. Optimizado para consultas analíticas.
Fact / Dimension	Hechos = eventos medibles (grabaciones). Dimensiones = el contexto (artista, standard, fecha).
Grano (grain)	Qué representa exactamente una fila de la tabla de hechos. La decisión de diseño clave.
Marts	Las tablas finales, limpias y modeladas, listas para que las consuma el dashboard.
Lineage	El mapa de dependencias entre modelos: qué tabla alimenta a cuál. dbt lo genera automático.
Tests (dbt)	Aserciones sobre los datos (not_null, unique, relaciones) que corren en cada build.

Mathias Berg — Data Analyst → Analytics Engineer · mbergr.github.io/portfolio · Documento generado para acompañar el repositorio jazz-standards-analytics.